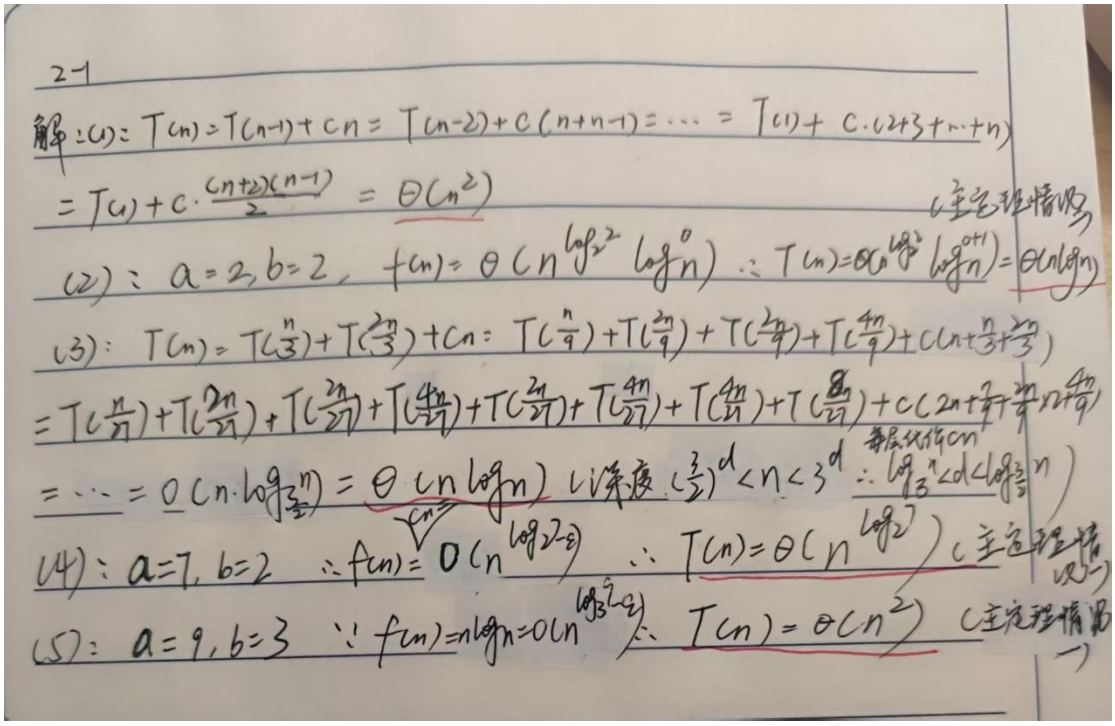
算法分析题

2-1



算法实现题

2-1 幂函数

1. 问题分析

（1）问题描述：给定一个实数 x 和一个整数 n（正负 0 都有可能），计算x^n的值。

（2）难点分析：正指数可能非常大，要注意时间复杂度；负指数处理同样要注意溢出以及精度控制。

2. 算法思想

采用快速幂的算法思想，利用二分法将指数分解，以减少乘法运算次数。

具体实现：

（1）n=0 时直接返回 1 无需进行递归。

（2）n 为负数的时候，计算x^{-n}的倒数从而转化为 n 为正数的情形。

（3）n 为正数的时候，递归计算 half=pow（x,n/2）,若 n 为偶数则结果即为 half*half；若为奇数则为 half*half*x（因为“/”具有向下取整特性）。

3. 算法伪代码（契合 leetcode 题干要求）

函数 pow(x, n):

输入: 实数 x, 整数 n (long long)

输出: x^n

如果 n == 0:

返回 1.0

否则如果 n < 0:

返回 1.0 / pow(x, -n)

否则:

half = pow(x, n / 2)

如果 n % 2 == 0:

返回 half * half

否则:

返回 half * half * x

函数 mypow(x, n):

输入: 实数 x, 整数 n (int)

输出: x^n

N = (long long) n

返回 pow(x, N)

主函数:

输入 x, n

输出 mypow(x, n)

4. 复杂度分析

（1）时间复杂度：每次递归调用将指数减半，因此递归深度为$log_2|n|+1$，每层进行常

数次乘法运算，故总时间复杂度为 O（$log\,|n|$），而当 n 为负数时，只需多一次倒数运算，不影响渐进复杂度。

（2）空间复杂度：递归调用采用栈的形式，栈深度为 O（$log\,|n|$），因此空间复杂度也为 O（$log\,|n|$）。

2-2 数组中第 k 个最大元素

1. 问题分析

（1）问题描述：给定一个整数数组 nums 和一个整数 k，要求返回数组中第 k 个最大的元素。(注：第 k 个最大元素指的是将数组按降序排序后位于第 k 个位置的元素（或按升序排序后位于第 n-k 个位置的元素，不是第 k 个不同的元素，即允许重复值。）同时设计的算法时间复杂度必须为 O（n）.

（2）难点分析：若采用普通排序后再查找的方法时间复杂度为 O（$n\,log\,n$）不符合要求，同时还要注意数组重复元素的问题。

2. 算法思想

采用随机化快速选择算法，基于快速排序的划分思想，只递归处理包含目标元素的那一侧。在代码中，采用两路划分（双指针）的方式，将数组分为“小于 pivot”和“大于等于 pivot”两部分。经过一次划分后，pivot 最终位置 pos 即它在排序后数组中的索引。若 pos 等于目标索引，则直接返回该元素；若 pos 小于目标索引，则在右半部分继续查找；否则在左半部分查找。

同时，为规避最坏情况（$O(n^2)$），采用随机选取 pivot 的算法，在区间 [left, right] 内随机选择一个下标，将其元素作为 pivot 并交换到区间起始位置，保证划分的期望平衡。

3. 算法伪代码

```
// 划分函数
// 输入：数组 nums，子数组下标范围 [a, b]（以 nums[a] 为 pivot）
// 输出：pivot 在排序后应处的位置
function partition(nums, a, b):
    pivot = nums[a]
    i = a
```

```
    j = b + 1
    while true:
        // 从左向右找到第一个不小于 pivot 的元素
        i = i + 1
        while i < b and nums[i] < pivot:
            i = i + 1
        // 从右向左找到第一个不大于 pivot 的元素
        j = j - 1
        while j > a and nums[j] > pivot:
            j = j - 1
        if i >= j:
            break
        // 交换这两个元素
        swap(nums[i], nums[j])
    // 将 pivot 放到正确位置
    swap(nums[a], nums[j])
    return j

// 随机化划分函数
// 在 [a, b] 中随机选择 pivot，交换到首位，然后调用 partition
function randompartition(nums, a, b):
    c = 随机整数，范围 [a, b]
    swap(nums[a], nums[c])
    return partition(nums, a, b)

// 随机化快速选择函数
// 在 nums[a..b] 中查找第 target 小的元素（target 为全局排序后的索引）
```

```
function randomquickselect(nums, a, b, target):
    if a == b:
        return nums[a]
    pos = randompartition(nums, a, b)    // 划分后 pivot 的最终位置
    if pos == target:
        return nums[pos]
    else if pos < target:
        // 目标在右半部分
        return randomquickselect(nums, pos+1, b, target)
    else:
        // 目标在左半部分
        return randomquickselect(nums, a, pos-1, target)

// 主函数：返回第 k 大的元素
function findKthLargest(nums, k):
    n = nums 的长度
    target = n - k
    return randomquickselect(nums, 0, n-1, target)
```

4. 复杂度分析

（1）时间复杂度

每次 partition 操作需要线性扫描当前子数组，时间复杂度为 O(m)，其中 m 为当前区间长度。在随机 pivot 的期望下，每次划分后问题规模大致减半，递归深度为 O(log n)，因此期望时间复杂度满足$T(n) = T(n/2) + O(n)$，根据主定理分析可知，时间复杂度在一般情况下都为 O（n），

（注：最坏情况下每次 pivot 选到最小或者最大元素为 O（n^2）的时间复杂度，但由于本算法采用了随机选取的策略，出现此情况的概率极低，，因此满足期望复杂度为 O（n））.

（2）空间复杂度

递归实现时，系统栈深度在期望情况下为 O(log n)，最坏情况下为 O(n)。每层递归仅使用常数个额外变量，因此期望空间复杂度为 O(log n)。

2-3 漂亮数组

1. 问题分析

① 问题描述：给定整数 n，要求构造一个长度为 n 的数组 nums，满足以下条件：（保证任意一个 n 都至少有一组解）

（1）nums 数组是 1~n 的一个排列（每个数字有且仅出现一次）

（2）不存在三个索引 i<k<j，使得 nums[k]是 nums[i]和 nums[j]的算术平均数。

② 难点分析：由于随着 n 增大，排列总数呈指数级增长，直接构造满足条件的排列极为困难，因此需要利用数学性质以及递归分治思想简化问题。

2. 算法思想

采用分治法，核心基于以下数学分析：

如果数组 A 是一个漂亮数组，那么对 A 中每个元素乘以 2 再减去 1，得到的新数组 2A−1 仍然是一个漂亮数组，且所有元素为奇数；对 A 中每个元素乘以 2，得到的新数组 2A 仍然是一个漂亮数组，且所有元素为偶数。将上述两个漂亮数组拼接（奇数在前，偶数在后），得到的新数组仍然是漂亮数组。

接下来进行相应数学证明，因为构造的奇数部分数组经过乘 2 再减 1 的构造一定小于构造的偶数部分数组乘 2 的任意元素（本身奇偶块间排列有序），即拼接后的所有奇数在所有偶数之前，现在考虑元素选取，若三个元素均在奇数块或者偶数块，由递归性质一定不违反条件；否则，由于必须满足 i<k<j，那么 i 所在位置元素一定在奇数块同时 j 所在元素一定在偶数块，但此时这两个元素加和为奇数，一定不满足题目中的等式，综上，一定能找到这样一个漂亮数组。

具体构造步骤：、

（1）n=1 时直接返回[1]（非常关键！！否则会导致无限递归）

（2 递归构造长度为 n/2 的漂亮数组 left 和 right（奇数和偶数部分各一个）

（3 将 left 中的每个元素乘 2 后减 1 进行映射，right 中元素乘 2 进行映射，然后按顺序拼接得到最终漂亮数组。

3. 算法伪代码

函数 beautifulArray(n):

输入: 整数 n

输出: 一个长度为 n 的漂亮数组

如果 n == 1:

    返回 [1]

否则:

    // 递归构造左右两部分

    left = beautifulArray((n+1)/2)   // 向上取整

    right = beautifulArray(n/2)   // 向下取整

    // 初始化结果数组

    result = 空数组

    // 将 left 中的每个元素映射为奇数，加入 result

    for x in left:

        result.append(2*x - 1)

    // 将 right 中的每个元素映射为偶数，加入 result

    for y in right:

        result.append(2*y)

    return result

4. 复杂度分析

（1）时间复杂度：设 T（n）为构造长度为 n 的漂亮数组所需时间，左右两边各 n/2 分别进行递归调用，并用 O（n）的线性时间进行左右合并操作，$T(n) = 2T\left(\frac{n}{2}\right) + O(n)$，由主定理可知，时间复杂度为 O（nlogn）

（2）空间复杂度：递归调用栈深度为 O（logn），每层占用常数空间，每次递归返回的数组需要存储结果，最终结果长度为 n。中间结果在合并后会释放，但递归过程中同时存在多个数组副本（如左右子数组），因此空间复杂度为 O（nlogn）.

2-4 主元素问题

1. 问题分析

设 T[0.n-1]是 n 个元素的数组。对任一元素 x,设 S(x)={T[]=x)。要求判断是否存在一个元素 x 使得其出现次数|S(x)|>n/2（即在数组中出现次数超过一半）。

要求在三种不同情形下进行解答:

（1）数组有序时采用分治策略使用线性时间算法进行实现。

（2）数组无序时设计一个时间复杂度 O（nlogn）的算法实现。

（3）数组无序时设计一个线性时间算法实现。

2. 算法思想

（1）有序数组

若数组有序，且主元素存在，则主元素必位于中位数位置（下标 n/2)。(可用反证法证明）因此算法先取中间元素 choice = arr[n/2]，再通过二分查找确定该元素第一次出现的位置 start 和最后一次出现的位置 end，从而得到出现次数 length = end - start + 1，最后判断 length > n/2 是否成立。

（2）无序数组，O（nlogn）时间复杂度

先采用时间复杂度为 O（nlogn）的快速排序方法变成有序数组，再使用跟（1）相同方法进行实现。

（3）无序数组，O（n）时间复杂度

采用摩尔投票算法，第一遍扫描维护一个候选元素 candidate 和一个计数器 count。遍历数组，若 count == 0，则设置当前元素为候选；若当前元素等于候选，count++，否则 count--。第二遍扫描: 统计候选元素实际出现的次数，若大于 n/2 则存在主元素，否则不存在。该算法利用“多数元素若存在，则抵消后必然留下该元素”的原理，可巧妙实现线性的时间复杂度

3. 算法伪代码

（1）有序数组

算法 hasmajor(arr)

输入：有序数组 arr[0..n-1]

输出：若存在主元素则返回 true，否则返回 false

1. n ← length(arr)

2. if n = 0 then return false

3. choice ← arr[⌊n/2⌋]

4. start ← findfirst(arr, choice, 0, n-1)

5. end ← findlast(arr, choice, 0, n-1)

6. if end - start + 1 > n/2 then return true

7. else return false

函数 findfirst(arr, x, l, r)

// 返回 x 在有序数组 arr[l..r] 中第一次出现的位置，若不存在则返回 -1

1. if l > r then return -1

2. mid ← ⌊(l + r)/2⌋

3. if arr[mid] = x then

4. pos ← findfirst(arr, x, l, mid-1)

5. if pos = -1 then return mid

6. else return pos

7. else if arr[mid] < x then

8. return findfirst(arr, x, mid+1, r)

9. else

10. return findfirst(arr, x, l, mid-1)

函数 findlast(arr, x, l, r)

// 返回 x 在有序数组 arr[l..r] 中最后一次出现的位置，若不存在则返回 -1

1. if l > r then return -1

2. mid ← ⌊(l + r)/2⌋

```
3. if arr[mid] = x then
4.         pos ← findlast(arr, x, mid+1, r)
5.         if pos = -1 then return mid
6.         else return pos
7. else if arr[mid] < x then
8.         return findlast(arr, x, mid+1, r)
9. else
10.        return findlast(arr, x, l, mid-1)
```

（2）无序数组，O（nlogn）时间复杂度

算法 hasmajor2(arr)

输入：无序数组 arr[0..n-1]

输出：若存在主元素则返回 true，否则返回 false

```
1. n ← length(arr)
2. quicksort(arr, 0, n-1)                    // 将 arr 按非降序排序
3. choice ← arr[⌊n/2⌋]
4. start ← findfirst(arr, choice, 0, n-1)
5. end ← findlast(arr, choice, 0, n-1)
6. if end - start + 1 > n/2 then return true
7. else return false
```

算法 quicksort(arr, a, b)

// 对 arr[a..b] 进行快速排序

```
1. if a ≥ b then return
2. c ← partition(arr, a, b)
3. quicksort(arr, a, c-1)
4. quicksort(arr, c+1, b)
```

算法 partition(arr, a, b)

// 快速排序划分过程，返回枢轴最终位置

1. i ← a, j ← b+1

2. x ← arr[a]

3. while true do

4. repeat i ← i+1 until arr[i] ⩾ x 或 i = b

5. repeat j ← j-1 until arr[j] ⩽ x

6. if i ⩾ j then break

7. swap(arr[i], arr[j])

8. swap(arr[a], arr[j])

9. return j

（3）无序数组，O（n）时间复杂度

算法 hasmajor3(arr)

输入：无序数组 arr[0..n-1]

输出：若存在主元素则返回 true，否则返回 false

1. n ← length(arr)

2. if n = 0 then return false

3. count ← 0, candidate ← 0

4. for i ← 0 to n-1 do

5. if count = 0 then candidate ← arr[i]

6. if arr[i] = candidate then count ← count + 1

7. else count ← count - 1

8. count2 ← 0

9. for i ← 0 to n-1 do

10. if arr[i] = candidate then count2 ← count2 + 1

11. if count2 > n/2 then return true

12. else return false

4. 复杂度分析

算法	时间复杂度	空间复杂度	说明
有序分治	$O(\log n)$	$O(\log n)$ 递归栈	仅依赖于两次二分查找，递归深度为 $\log n$。
无序快排+分治	平均 $O(n\log n)$	平均 $O(\log n)$ 递归栈	快速排序的平均性能为 $O(n\log n)$，加上算法一的有序分治$O(n)$，总时间复杂度仍为 $O(n\log n)$
无序摩尔投票	$O(n)$	$O(1)$	仅需两次线性扫描，均为O（n）复杂度，常数空间，且不依赖数组的有序性，为最优解法。